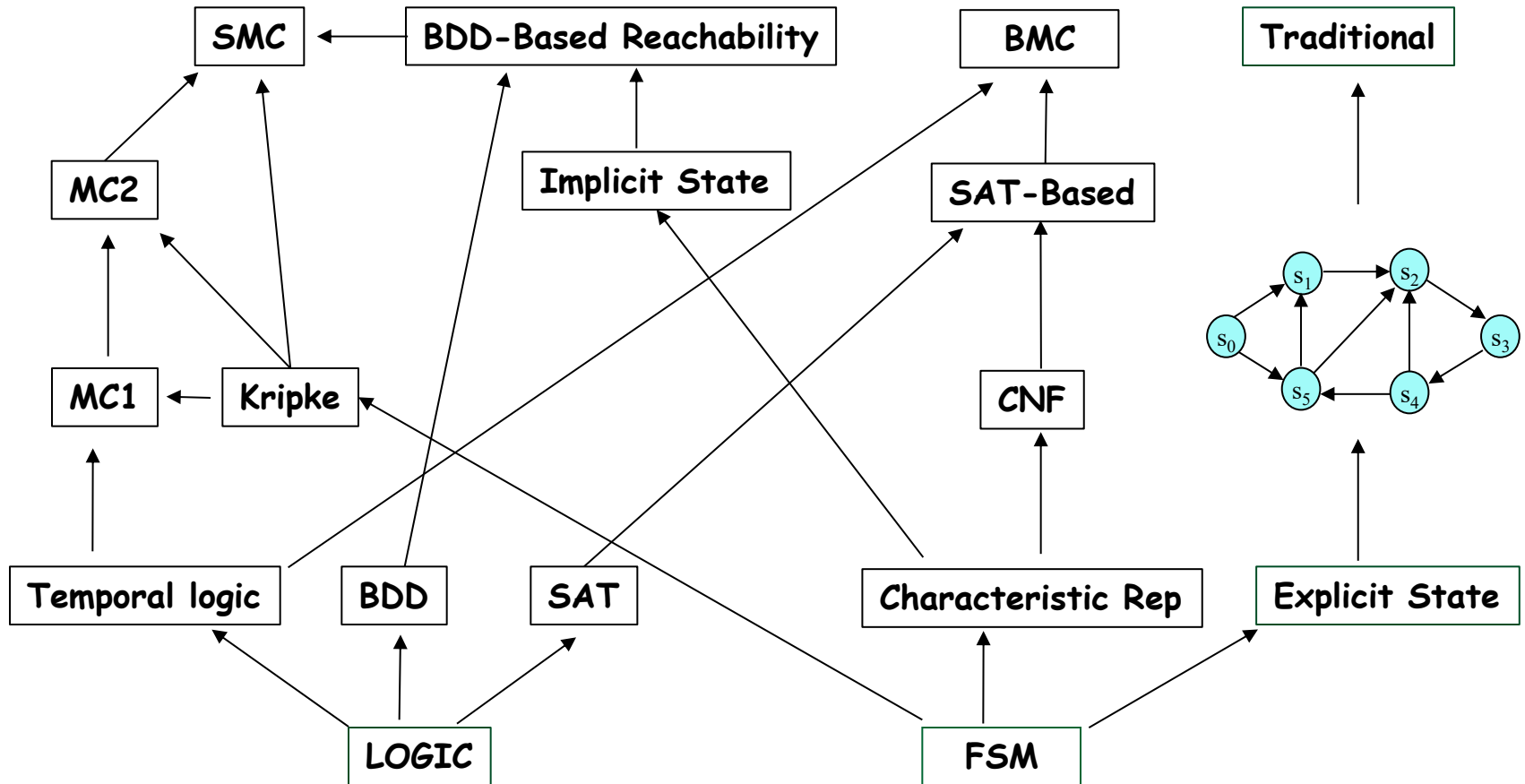




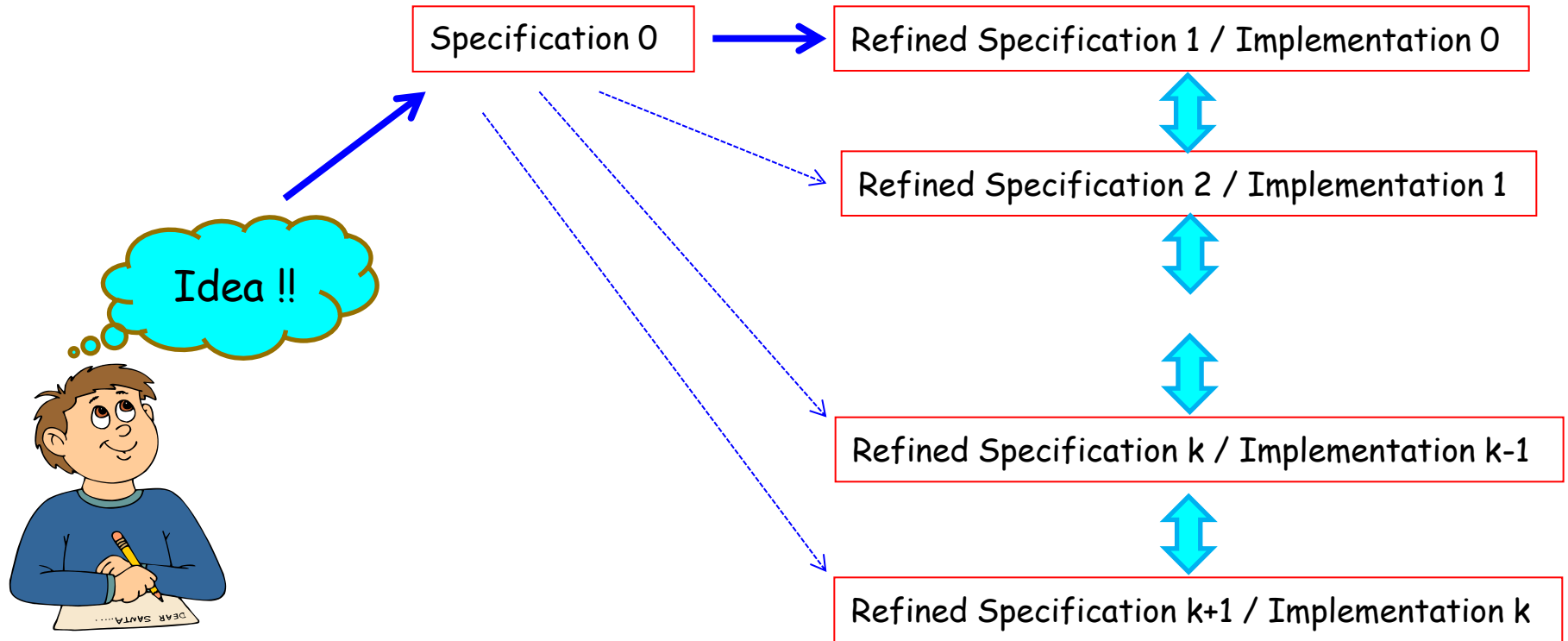
Propositional Logic



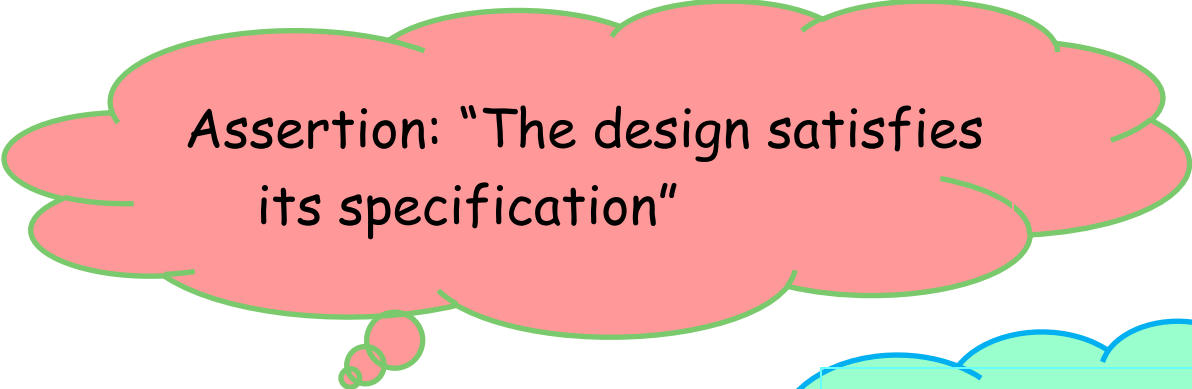
A Road Map



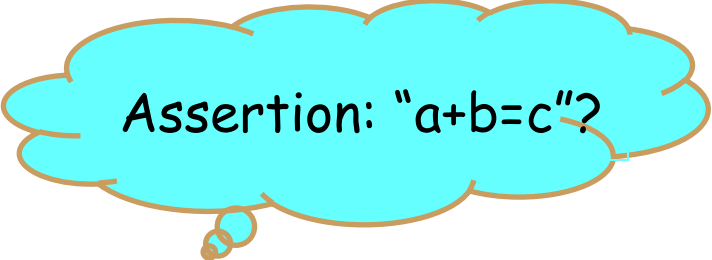
Modeling and Verification



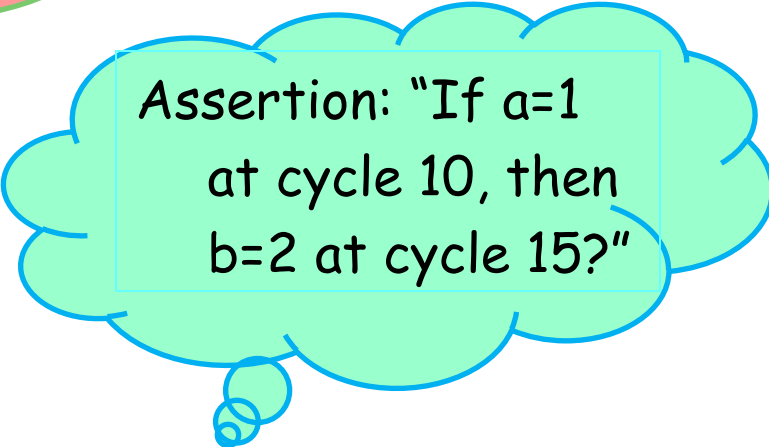
Basic Verification Questions



Assertion: "The design satisfies its specification"



Assertion: " $a+b=c$ "?

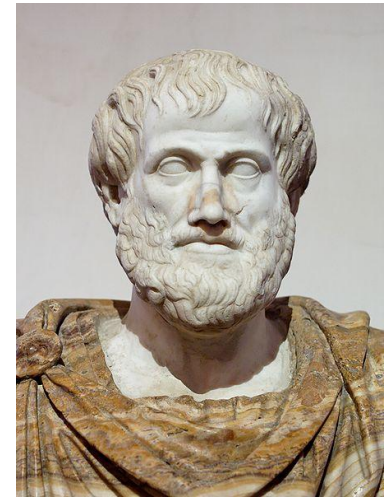


Assertion: "If $a=1$ at cycle 10, then $b=2$ at cycle 15?"

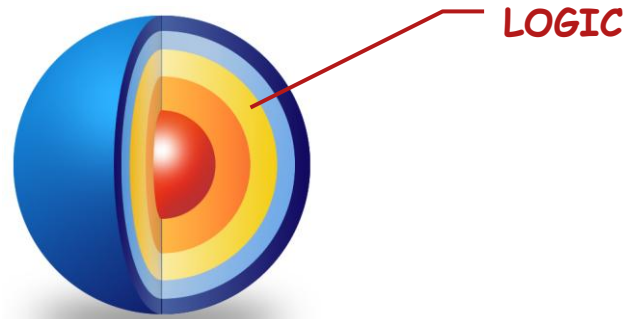
- Interested in knowing when/how assertions are true/false.
- Is there any study on the truth of sentences in general?

Why Logic Involved?

- The answer is Yes: the topic is Logic.
- Logic is the study of arguments.
- Logic was established as a discipline by Aristotle (384—322 BCE).
 - Averroes: "the tool for distinguishing between the true and the false;"
 - Richard Whately: "the Science, as well as the Art, of reasoning;"
 - Gottlob Frege: "the science of the most general laws of truth."
- <http://en.wikipedia.org/wiki/Logic>



Logics in Computing Systems



- Logic is a fundamental organizing principle in computing systems.
- It runs a multifaceted gamut from the foundational to the applied.
- At one extreme,
 - it underlies computability and complexity theory and the formal semantics of programming languages.
- At the other,
 - it drives billions of gates every day in the digital circuits of processors of all kinds.

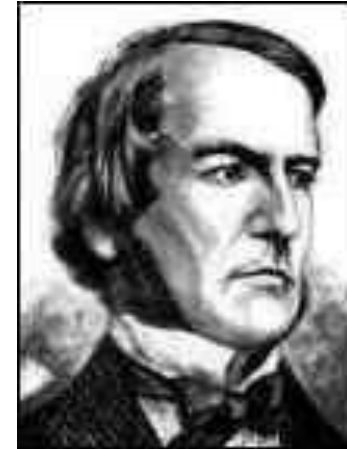
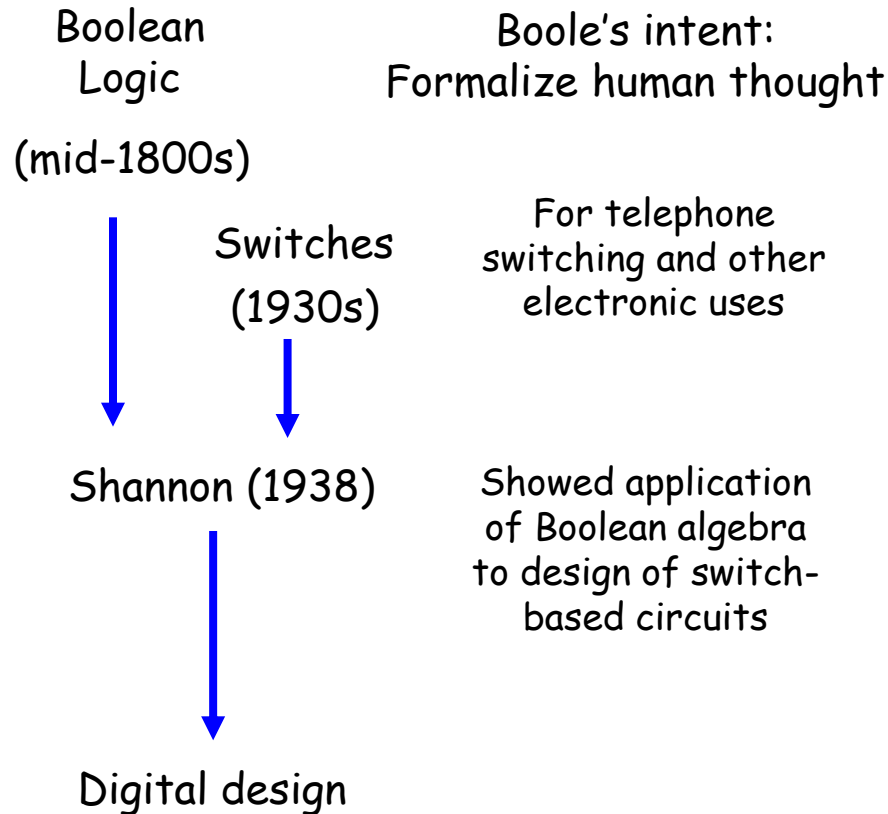
Logic in Computing Systems

- Logic is in itself a powerful programming paradigm but it is also the quintessential specification language.
- Logical techniques link implementation and specification through formal methods
 - such as automated theorem proving and model checking.
- Logic is the knowledge representation and artificial intelligence.
- Because of its ubiquity, logic has acquired a central role in computing education.



Gödel 1931

Boolean Logic ?



- George Boole (1815 - 1864)
- Boolean logic can be used
 - not only for the combinational circuit design,
 - but also for **reasoning**

What is a Logic?

- A logic is
 - a language related to the study of truth
 - the principles of valid demonstration and inference.
- The principal question in a logic is
 - to answer if a logic statement is true or false.

Propositional Logic (Boolean Logic)

- A proposition is a sentence that can be either true or false.
 - Propositions:
 - x is greater than y
 - Paris is the capital of France
- Propositional logic is a logic about truth and falsity of sentences.

Syntax: Atomic Propositions

- The **smallest unit** of propositional logic is an **atomic** sentence.
 - Such atomic components can be either *true* or *false*, in our understanding of the world.
 - No analysis will be done to the components of an atomic sentence.
- Atomic sentences which are true or false are basic propositions.

Syntax: Atomic Propositions = Variables

- For example, such atomic sentences are:
 - A = “Paris is the capital of France”
 - B = “Mice chase elephants”
- A, B are abstractions of “atomic sentences”
 - called = “Boolean variables”, or “variables”

Syntax: Composite Propositions

- Larger and more complex sentences are constructed from atomic propositions by combining them using connectives.
- Thus, **atomic propositions and connectives** are the basic elements of propositional logic.
- Propositional logic is a language
 - simple grammatical connections:
 - *and, or, and not*, between the simplest “atomic sentences”.

Syntax: Connectives

- P, Q, R : Propositional symbols (variables)
- $\neg P$: **Negation** of P
- $P \wedge Q$: Conjunction, P **and** Q
- $P \vee Q$: Disjunction, P **or** Q
- $P \rightarrow Q$: Implication, Conditional, **if** P **then** Q
- $P \leftrightarrow Q$: Equivalence, biconditional, P **if and only if** Q

Semantics: Truth Values = {true, false}

- If a proposition is true,
 - then we say it has a truth value of "true";
- If a proposition is false,
 - its truth value is "false".
- We are only interested in true or false sentences, but not both.

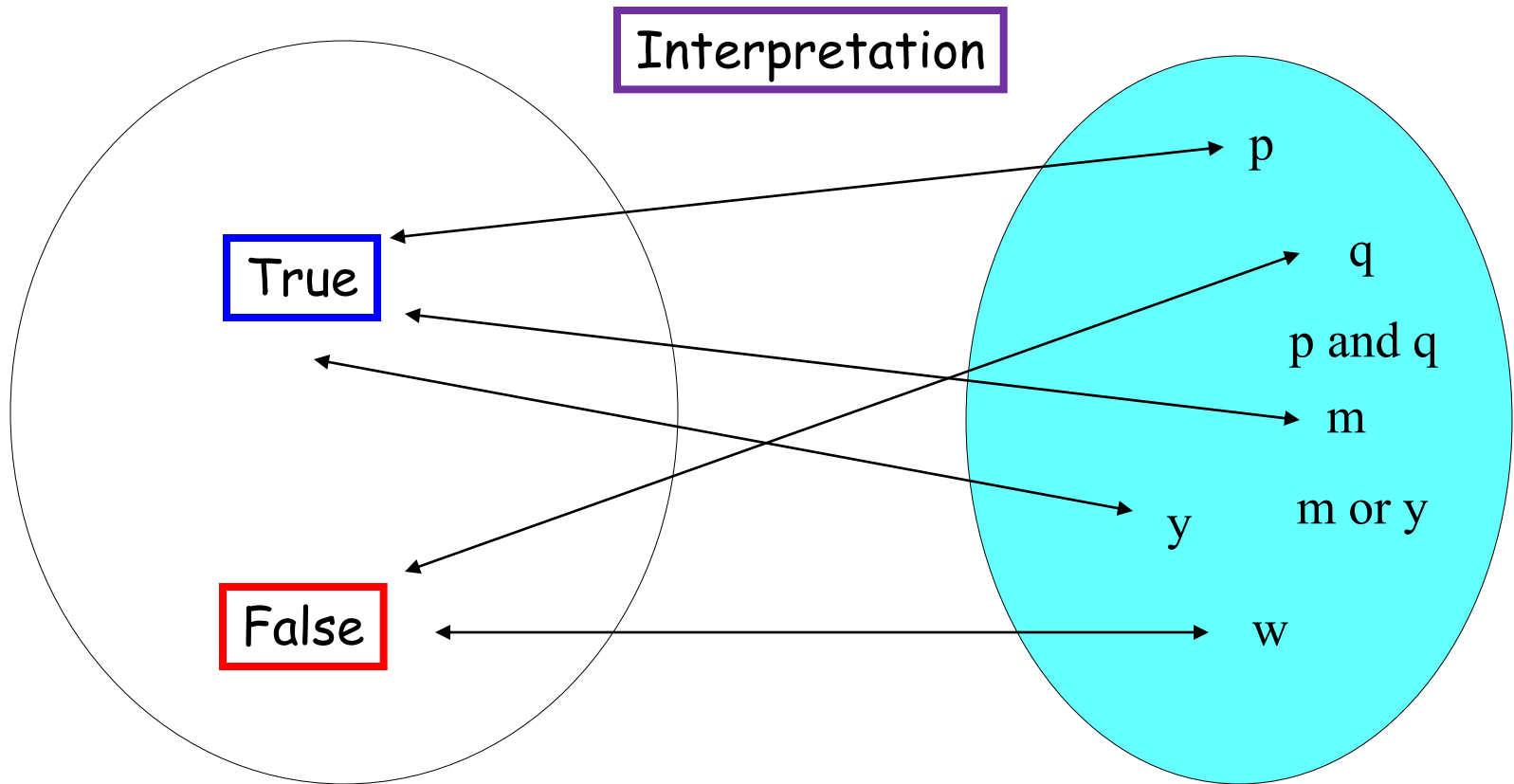
Propositional Logic

- The subject of propositional logic is to
 - declare formally how such “truth values” of the atomic components extend to a truth value of a more complex structure, such as *A and B*.

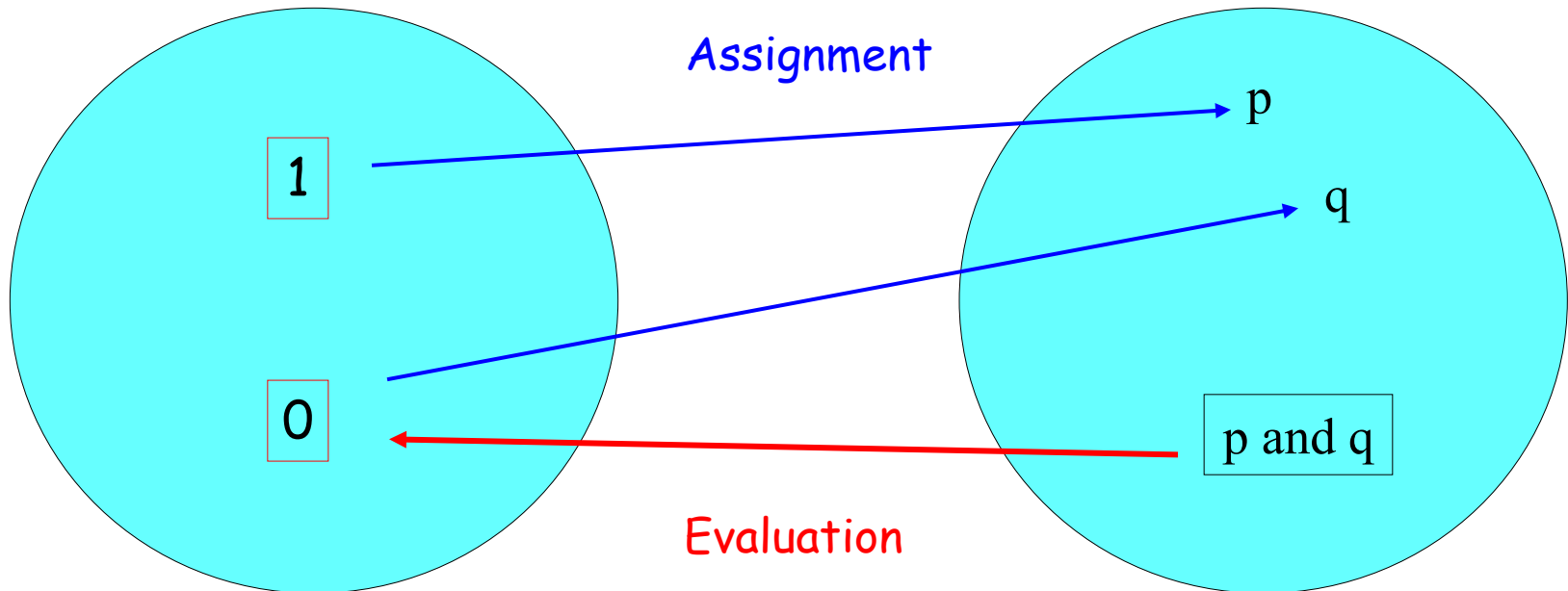
Language: Syntax & Semantics

Semantic Domain

Syntactic Domain



Interpretation



- Interpretation = Assignment + Evaluation
- A truth-value assignment I is a mapping: $\{\text{propositional symbols}\} \rightarrow \{1, 0\}$.
- Evaluation: Truth table and Computing

Truth Table: Simple Interpretations

	P	Q	$P \wedge Q$
I1	0	0	0
I2	0	1	0
I3	1	0	0
I4	1	1	1

- Truth Table: a set of simple interpretations: I1, I2, ...etc.
 - only for **primitive** expressions
 - Assignment + **Trivial** Evaluation

Truth Table: Simple Interpretations

	P	$\neg P$
I1	0	1
I2	1	0

	P	Q	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
I1	0	0	0	1	1
I2	0	1	1	1	0
I3	1	0	1	0	0
I4	1	1	1	1	1

- Truth Table:
 - only for **primitive** expressions
 - = Assignment + **Trivial** Evaluation

Key Concepts with Interpretations

- Use interpretations in stead of truth tables
- Formula F is **satisfiable** (consistent)
 - iff it is true under at least one interpretation.
- Formula F is **unsatisfiable** (inconsistent)
 - iff it is false under all interpretations.
- Formula F is **valid**
 - iff it is true (consistent) under all interpretations.

The Mirror Principle

Valid formulas	Satisfiable, but non-valid formula	Unsatisfiable formulas
G	$F < \text{---} > \neg F$	$\neg G$

Interpretations: $\models$ and $\not\models$

	P	Q	$P \wedge Q$
I1	0	0	0
I2	0	1	0
I3	1	0	0
I4	1	1	1

- I1 $\not\models P \wedge Q$
- I2 $\not\models P \wedge Q$
- I3 $\not\models P \wedge Q$
- I4 $\models P \wedge Q$

- Interpretation I **satisfies** a formula F iff F is true under I
 - I $\models$ F
 - I is a **model** of F

Interpretation: $\models$

- I is a **model** of F iff F is true under I $\models$ F.

	P	Q	$P \wedge Q$
I1	0	0	0
I2	0	1	0
I3	1	0	0
I4	1	1	1

- I1 $\not\models P \wedge Q$
- I2 $\not\models P \wedge Q$
- I3 $\not\models P \wedge Q$
- I4 $\models P \wedge Q$

- A truth assignment $\tau : \text{Prop} \rightarrow \{0,1\}$
- $\tau \models p$ means
 - τ satisfies p
 - τ is true of p
 - p holds at τ
 - τ is a model of p

Evaluation for Complex Expressions

P	Q	$P \wedge Q$
0	0	0
0	1	0
1	0	0
1	1	1



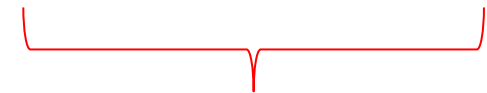
- $I1 \models P \wedge Q ?$
- $I2 \models P \wedge Q ?$
- $I3 \models P \wedge Q ?$
- $I4 \models P \wedge Q ?$

f	g	d	b	p	q	M
0	0	1	0	0	1	
0	1	1	0	0	1	?
1	0	0	0	1	0	
1	1	0	1	1	1	

τ

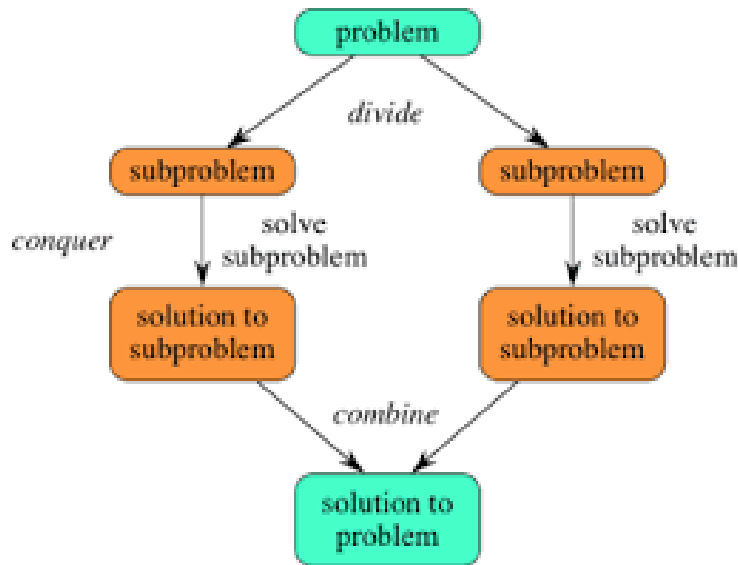
?

$\tau \models (f \vee g) \wedge (d \vee ((b \vee q) \wedge p)) ?$



Computation

The Recursive Truth Table



- $\tau \models A$ if $\tau(A) = 1$ (or, $A \in \tau$)
- $\tau \models \neg p$ if it is not the case $\tau \models p$.
- $\tau \models p \wedge q$ if $\tau \models p$ and $\tau \models q$
- $\tau \models p \vee q$ if $\tau \models p$ or $\tau \models q$
- $\tau \models p \rightarrow q$ if $\tau \models p$ implies $\tau \models q$
- $\tau \models p \leftrightarrow q$ if $\tau \models p$ iff $\tau \models q$

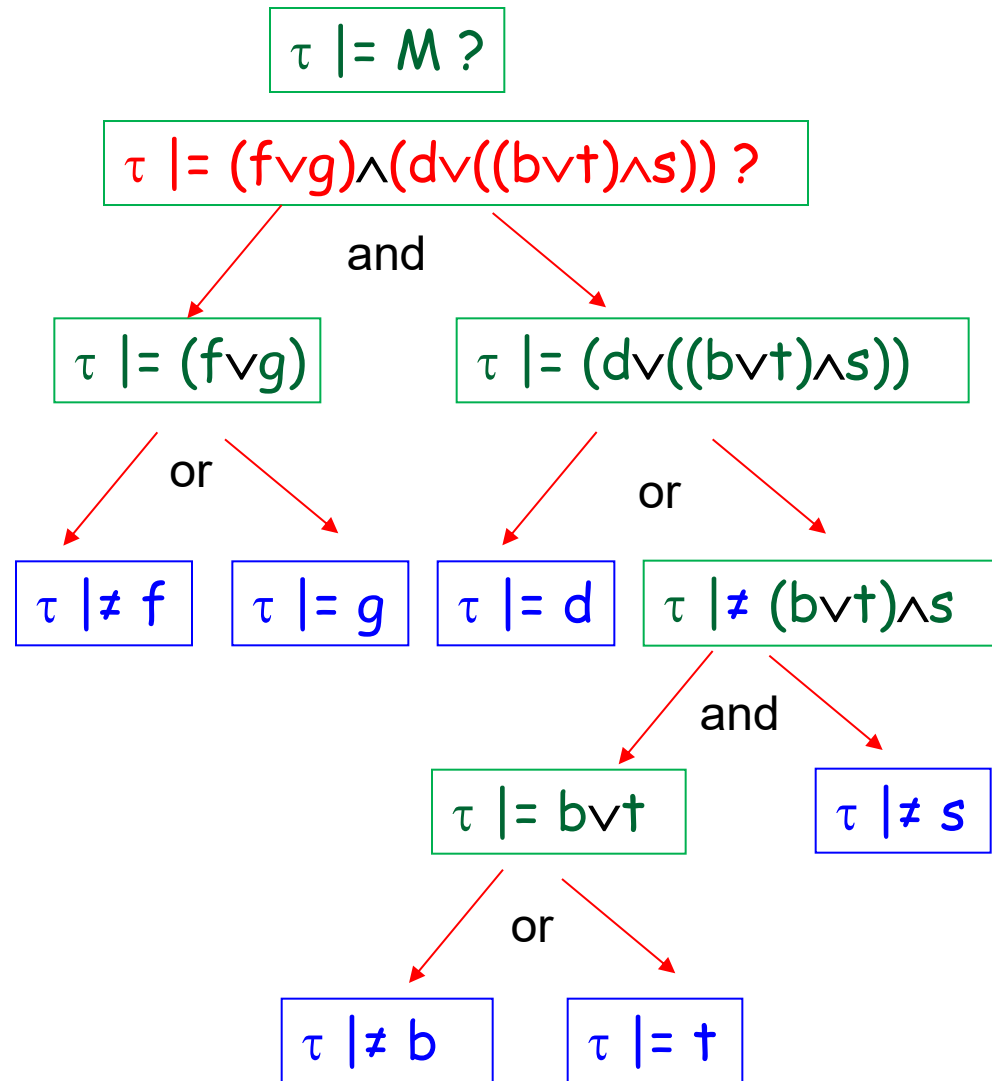
- A divide-and-conquer algorithm works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.

Recursive Divide-and-Conquer Evaluation

- $\tau \models A$ if $\tau(A) = 1$ (or, $A \in \tau$)
- $\tau \models \neg p$ if it is not the case $\tau \models p$.
- $\tau \models p \wedge q$ if $\tau \models p$ and $\tau \models q$
- $\tau \models p \vee q$ if $\tau \models p$ or $\tau \models q$
- $\tau \models p \rightarrow q$ if $\tau \models p$ implies $\tau \models q$
- $\tau \models p \leftrightarrow q$ if $\tau \models p$ iff $\tau \models q$

τ

f	g	d	b	s	t	M
0	0	1	0	0	1	1
0	1	1	0	0	1	1
1	0	0	0	1	0	0
1	1	0	1	1	1	1



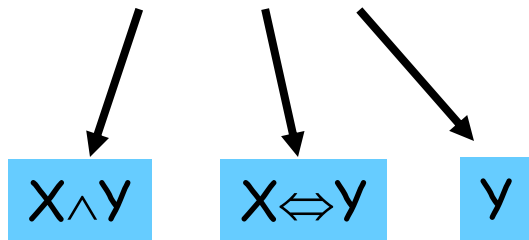
Many ways to say: $A \rightarrow B$

P	Q	$P \rightarrow Q$
0	0	1
0	1	1
1	0	0
1	1	1

- If A, then B.
- A implies B.
- A, therefore B.
- A only if B.
- B follows from A.
- B whenever A
- B if A
- A is a sufficient condition for B
- B is a necessary condition for A.

Implication: $X \rightarrow Y$

X	Y	T1	T2	T3	T4
0	0	0	1	0	1
0	1	0	0	1	1
1	0	0	0	0	0
1	1	1	1	1	1



$$X \rightarrow Y = \neg X \vee Y$$

- If α is false, then $\alpha \rightarrow \beta$ is regarded as true, whether or not there is any connection between the falsity of α and the truth-value of β .
- $\alpha \rightarrow \beta$
 - α is called the antecedent,
 - β is called the consequent.

Many Notions on Implication

- Conditional, Converse, Inverse, and Contrapositive

				Conditional	Converse	Inverse	Contrapositive
P	Q	$\sim P$	$\sim Q$	$P \rightarrow Q$	$Q \rightarrow P$	$\sim P \rightarrow \sim Q$	$\sim Q \rightarrow \sim P$
T	T	F	F	T	T	T	T
T	F	F	T	F	T	T	F
F	T	T	F	T	F	F	T
F	F	T	T	T	T	T	T

Conditional & Biconditional

P	Q	$P \rightarrow Q$	$P \leftrightarrow Q$
0	0	1	1
0	1	1	0
1	0	0	0
1	1	1	1

- Question:

- Can you use $P \rightarrow Q$ to express $P \leftrightarrow Q$?

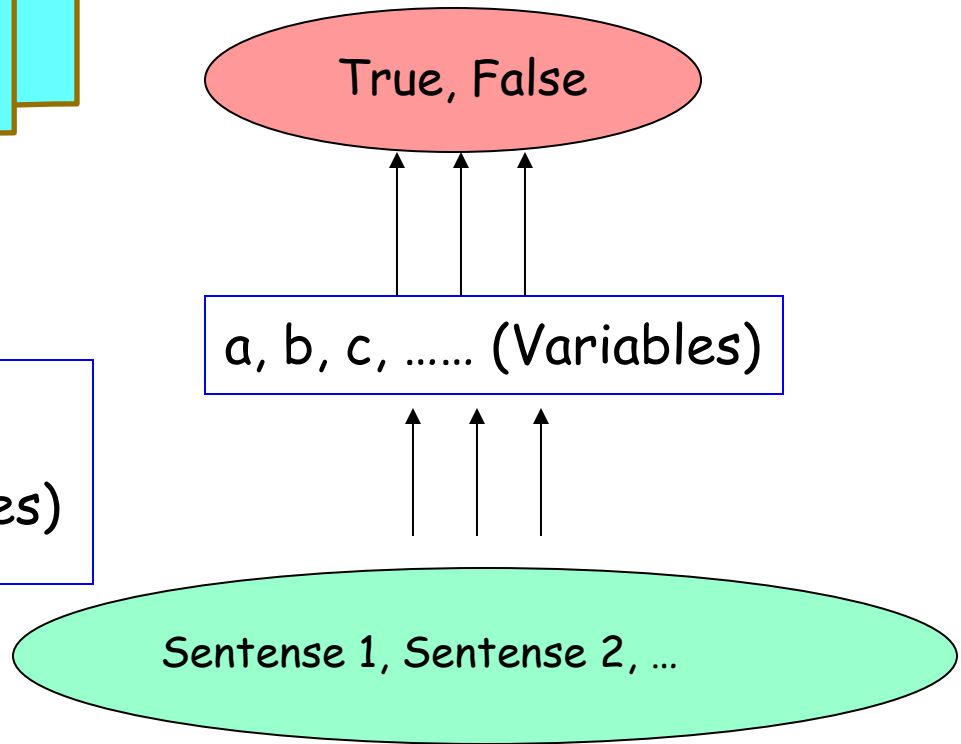
Defining Set Theory in Boolean Logic

- $A \cap B$ means
 - $\forall x (x \in A \cap B) \rightarrow (x \in A) \wedge (x \in B).$
- $A \cup B$ means
 - $\forall x (x \in A \cup B) \rightarrow (x \in A) \vee (x \in B).$
- $A \subseteq B$ means "A is a subset of B."
 - $A \subseteq B \text{ iff } \forall x (x \in A) \rightarrow (x \in B).$
- $A = B$ iff A and B have exactly the same elements.
 - iff $A \subseteq B$ and $B \subseteq A$
 - iff $\forall x ((x \in A) \rightarrow (x \in B))$ and $\forall x ((x \in B) \rightarrow (x \in A))$

Summary: Propositional Logic

**The Principal Question:
F=1?**

- Domain: $B = \{\text{True}, \text{False}\}$,
- Objects: $a, b, c, \dots$ (Variables)



The Fundamental Problem

- Given a formula $F(x_1, x_2, \dots, x_n)$, can we compute the answer on its validity ($\forall I. F=1$) or satisfiability ($\exists I. F=1$).
- What does a computation mean?
 - An algorithm.



Validity & Satisfiability

- A problem is decidable if there is an algorithm for deciding the truth of any proposition.
- The validity and satisfiability of Boolean logic is:
 - Decidable
 - The truth table provides a systematic (algorithmic) way.
 - It suffices to test finite number (2^n) of interpretations of the atomic formulas occurring in F .
- The satisfiability problem is "NP-complete".

One Way to Establish the Truth

F=1?

Model Theory
|=

	P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
I1	0	0	1	0	0	1	1
I2	0	1	1	0	1	1	0
I3	1	0	0	0	1	0	0
I4	1	1	0	1	1	1	1

Evaluation: F=1?

$$F = A \cdot (\overline{B + \overline{C \cdot D}}) + \overline{A} \cdot \overline{\overline{B} + E}$$

ABCDE=00000

$$\begin{aligned} F &= 0 \cdot (\overline{0 + \overline{0 \cdot 0}}) + \overline{0} \cdot \overline{\overline{0} + 0} = 0 \cdot (\overline{0 + 1 \cdot 0}) + \overline{0} \cdot \overline{1 + 0} \\ &= 0 \cdot (\overline{0 + 0}) + 1 \cdot \overline{1} = 0 \cdot 1 + 1 \cdot 0 = 0 \end{aligned}$$

ABCDE=10000

$$\begin{aligned} F &= 1 \cdot (\overline{0 + \overline{0 \cdot 0}}) + \overline{1} \cdot \overline{\overline{0} + 0} = 1 \cdot (\overline{0 + 1 \cdot 0}) + 0 \cdot \overline{1 + 0} \\ &= 1 \cdot (\overline{0 + 0}) + 0 \cdot \overline{1} = 1 \cdot 1 + 0 \cdot 0 = 1 \end{aligned}$$

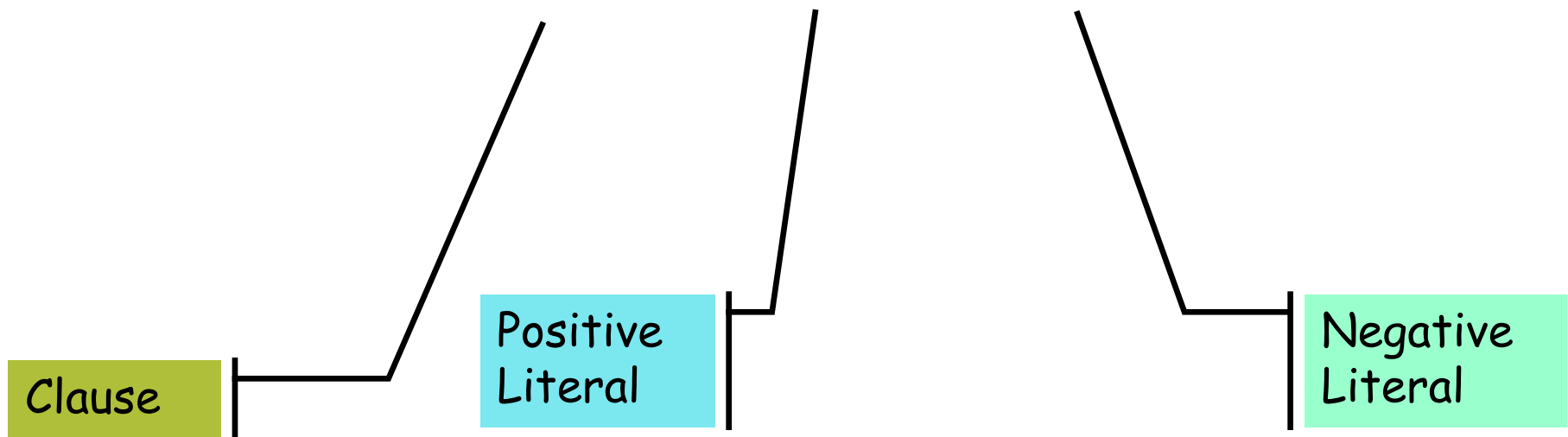
Normal Forms

- Let $A_1, A_2, \dots$ be atomic formula.
- **Literal** L is an atomic formula A or its negation $\neg A$
 - $L_{i,j} \in \{A_1, A_2, \dots\} \cup \{\neg A_1, \neg A_2, \dots\}$
- Conjunctive Normal Form (**CNF**): **POS**
 - Conjunction of disjunctions of literals: $F = (\wedge (\vee L_{i,j}))$
- Disjunctive Normal Form (**DNF**): **SOP**
 - Disjunction of conjunctions of literals: $F = (\vee (\wedge L_{i,j}))$
- Theorem:
 - For each F , there is an equivalent CNF and an equivalent DNF.

Satisfiability Problem (SAT)

- Given a propositional formula in CNF, find an assignment to Boolean variables that makes the formula true!

$$f = (a + c) (b + c) (\neg a + \neg b + \neg c)$$



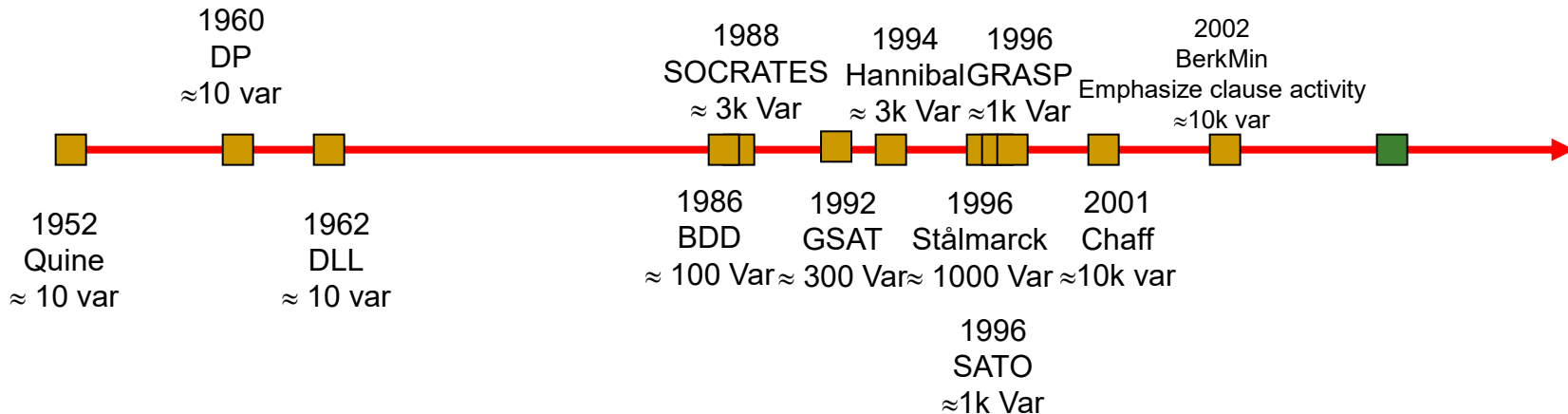
Satisfiability Problem (SAT)

- Solution 1: Search through all assignments!
 - n variables $\rightarrow 2^n$ possible assignments, **explosion!**
- SAT is a classic NP-Complete problem, solve SAT and P=NP!
- Fundamental problem from theoretical point of view
- Numerous applications
 - AI, planning, automated deduction

Satisfiability (SAT) Solver



The solver reports if F is satisfiable or unsatisfiable.
If satisfiable, at least one assignment is reported.



How to use a SAT Solver?

- Convert your problem into the input of a SAT solver
 - Problem Logic formulation for SAT
 - If the SAT solver requires a CNF form, convert it into a CNF form.
 - Convert the CNF form into the input numeric form.
- Run the solver and get the answer.
- Interpret the answer
 - Unsatisfiable: Meaning?
 - Satisfiable: Meaning? Model instances ?

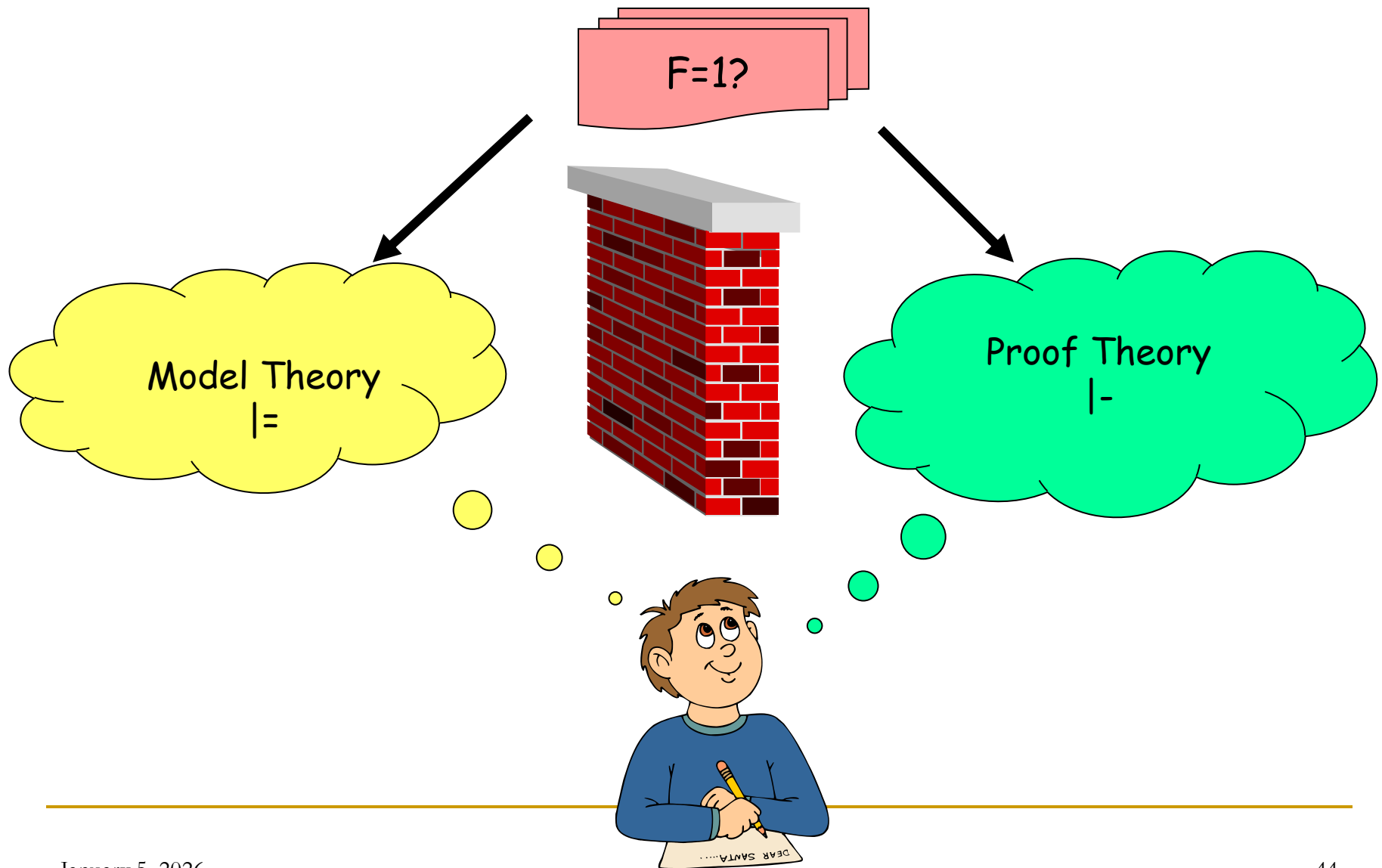
The Validity Problem by SAT Solvers

Validity Problem: $\forall I. F=1$

The solver reports if F is satisfiable or unsatisfiable.
If satisfiable, at least one assignment is reported.

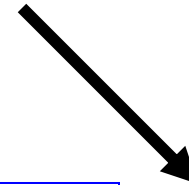
Valid formulas	Satisfiable, but non-valid formula	Unsatisfiable formulas
G	$F < \text{---} > \neg F$	$\neg G$

Two Ways to Establish the Truth



Another Way to Establish the Truth

F=1?



Proof Theory

|-

<u>Expression</u>	<u>Rule(s) Used</u>
$C + \overline{BC}$	Original Expression
$C + (\overline{B} + \overline{C})$	DeMorgan's Law.
$(C + \overline{C}) + \overline{B}$	Commutative, Associative Laws.
$T + \overline{B}$	Complement Law.
T	Identity Law.

Rules of Boolean Algebra

$X + 0 = X, X + 1 = 1$	(Identity)
$X + X = X$	(Idempotent Law)
$X + \bar{X} = 1$	(Complement)
$\overline{\bar{X}} = X$	(Involution Law)
$X + Y = Y + X$	(Commutative)
$X + (Y + Z) = (X + Y) + Z$	(Associative)
$X(Y + Z) = XY + XZ$	(Distributive)
$\overline{X + Y} = \bar{X}\bar{Y}$	(DeMorgan's Law)
$X + XY = X$	(Absorption Law)
$X + \bar{X}Y = X + Y$	(Absorption Law)
$XY + \bar{X}Z + YZ = XY + \bar{X}Z$	(Consensus Theorem)

$$X + \bar{X}Y$$

$$| - X \cdot (1 + Y) + \bar{X}Y$$

$$| - X + XY + \bar{X}Y$$

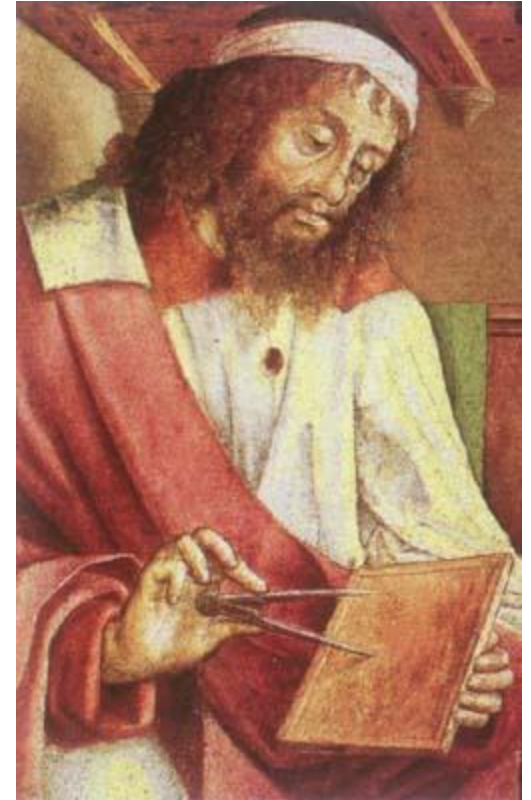
$$| - X + (X + \bar{X})Y$$

$$| - X + Y$$

$$\therefore X + \bar{X}Y = X + Y$$

Proofs

- A *proof* is a finite sequence of fixed indisputable steps.
- A proof is built from
 - *Axioms*:
 - facts which we accept without proof
 - Axioms are inference rules with no antecedents
 - Any instantiated axiom must be a tautology.
 - *Theorems*:
 - facts derived from axioms using an agreed-upon set of *rules of inference*.



Euclid (300 BC)

Proofs

- $\vdash$ is a relation defined by syntactic transformations of the underlying proof system.
 - Purely syntactic (mechanical) activity.
 - Not concerned with the meaning of statements.
- A proof is a sequence of statements where every statement follows from a preceding one by a rule of inference.
- A proof uses a given set of inference rules and axioms.
- This is called the proof system.

Modeling in Propositional Logic

- Let p , q , and r be the following propositions:
 - p : You get an A on the final exam
 - q : You do every exercise in the book.
 - r : You get an A in this class.
- Write the following formulas using p , q , and r and logical connectives.
 - You get an A in this class, but you do not do every exercise in the book.
 - To get an A in this class, it is necessary for you to get an A on the final.
 - Getting an A on the final and doing every exercise in the book is sufficient for getting an A in this class.

Modeling in Propositional Logic

Atomic sentence to be modeled by a symbol (variable)

- Assertion 1: You get an A in this class, but you do not do every exercise in the book.
 - $r \wedge q'$
- Assertion 2: To get an A in this class, it is necessary for you to get an A on the final.
 - $r \rightarrow p$
- Assertion 3: Getting an A on the final and doing every exercise in the book is sufficient for getting an A in this class.
 - $p \wedge q \rightarrow r$

Inference Rules

- Modus Ponens: $\{A \rightarrow B, A\} \vdash B$.

$$\frac{A \rightarrow B, \quad A}{B}$$

- $M \wedge N \vdash M$

A System Specification

- Specification:

- *If the operator types the ALARM command, then the subsidiary alarm will be activated and an alarm message written to the log file.*
- *Whenever the subsidiary alarm is activated, the main valve is closed and shut-down started.*
- *If the main valve is closed then the reactor can be regarded as being in a non-operational state.*

- Property: if the alarm command is activated the reactor will be in a non-operational state.

Modeling

■ Abstraction:

- alarm: *"the operator types the ALARM command"*
- subsid: *"the subsidiary alarm will be activated"*
- alarm_message: *"an alarm message written to the log file"*
- valve_closed: *"the main valve is closed"*
- shut_down: *"shut-down started"*
- non_op: *"the reactor can be regarded as being in a non-operational state"*

■ Modeling:

- $S_1: \text{alarm} \rightarrow (\text{subsid} \wedge \text{alarm_message})$
- $S_2: \text{subsid} \rightarrow (\text{valve_closed} \wedge \text{shut_down})$
- $S_3: \text{valve_closed} \rightarrow \text{non_op}$

Proof 1

- $S_1: \text{alarm} \rightarrow (\text{subsid} \wedge \text{alarm_message})$
- $S_2: \text{subsid} \rightarrow (\text{valve_closed} \wedge \text{shut_down})$
- $S_3: \text{valve_closed} \rightarrow \text{non_op}$
- $\{A \rightarrow B, A\} \vdash B.$

- Theorem: $\text{alarm} \vdash \text{non_op}.$

Proof: $(\text{alarm} \wedge S_1 \wedge S_2 \wedge S_3) \rightarrow \text{non_op}$

- $\vdash [\text{alarm} \wedge (\text{alarm} \rightarrow (\text{subsid} \wedge \text{alarm_message})) \wedge (\text{subsid} \rightarrow (\text{valve_closed} \wedge \text{shut_down})) \wedge S_3] \rightarrow \text{non_op}$
- $\vdash [(\text{alarm} \wedge \text{subsid} \wedge \text{alarm_message}) \wedge (\text{subsid} \rightarrow \text{valve_closed} \wedge \text{shut_down}) \wedge S_3] \rightarrow \text{non_op}$
- $\vdash [\text{alarm} \wedge \text{subsid} \wedge \text{alarm_message} \wedge \text{valve_closed} \wedge \text{shut_down} \wedge S_3] \rightarrow \text{non_op}$
- $\vdash [\text{alarm} \wedge \text{subsid} \wedge \text{alarm_message} \wedge \text{valve_closed} \wedge \text{shut_down} \wedge (\text{valve_closed} \rightarrow \text{non_op})] \rightarrow \text{non_op}$
- $\vdash \text{True}$

Proof 2: $\{A \rightarrow B, A\} \vdash B$.

- $S_1: \text{alarm} \rightarrow (\text{subsid} \wedge \text{alarm_message})$
- $S_2: \text{subsid} \rightarrow (\text{valve_closed} \wedge \text{shut_down})$
- $S_3: \text{valve_closed} \rightarrow \text{non_op}$
- $\{A \rightarrow B, A\} \vdash B. M \wedge N \vdash M$
- **Theorem:** $\text{alarm} \vdash \text{non_op}$.

Proof:

- | | |
|--|---|
| □ alarm | $\text{alarm} \rightarrow (\text{subsid} \wedge \text{alarm_message})$ |
| □ $\vdash \text{subsid} \wedge \text{alarm_message}$ | $\{A \rightarrow B, A\} \vdash B$ |
| □ $\vdash \text{subsid}$ | $M \wedge N \vdash M$ |
| □ $\vdash \text{valve_closed} \wedge \text{shut_down}$ | $\text{subsid} \rightarrow (\text{valve_closed} \wedge \text{shut_down})$ |
| □ $\vdash \text{valve_closed}$ | $M \wedge N \vdash M$ |
| □ $\vdash \text{non_op}$ | $\text{valve_closed} \rightarrow \text{non_op}$ |

A Complex Example

- If my glasses are on the kitchen table, then I saw them at breakfast.
- I was reading the newspaper in the living room or I was reading the newspaper in the kitchen.
- If I was reading the newspaper in the living room, then my glasses are on the coffee table.
- I did not see my glasses at breakfast.
- If I was reading my book in bed, then my glasses are on the bed table.
- If I was reading the newspaper in the kitchen, then my glasses are on the kitchen table.
- Where are the glasses?

Modeling

- If my glasses are on the kitchen table, then I saw them at breakfast.
 - $A =$ my glasses are on the kitchen table
 - $B =$ I saw my glasses at breakfast
 - $A \rightarrow B$
- I was reading the newspaper in the living room or I was reading the newspaper in the kitchen.
 - $C =$ I was reading the newspaper in the living room
 - $D =$ I was reading the newspaper in the kitchen.
 - $C \vee D$
- If I was reading the newspaper in the living room, then my glasses are on the coffee table.
 - $E =$ my glasses are on the coffee table
 - $C \rightarrow E$
- I did not see my glasses at breakfast.
 - B'
- If I was reading my book in bed, then my glasses are on the bed table.
 - $F =$ I was reading my book in bed
 - $G =$ my glasses are on the bed table
 - $F \rightarrow G$
- If I was reading the newspaper in the kitchen, then my glasses are on the kitchen table.
 - $D \rightarrow A$
- Where are the glasses?

Deduction Proof

- If my glasses are on the kitchen table, then I saw them at breakfast.
- I was reading the newspaper in the living room or I was reading the newspaper in the kitchen.
- If I was reading the newspaper in the living room, then my glasses are on the coffee table.
- I did not see my glasses at breakfast.
- If I was reading my book in bed, then my glasses are on the bed table.
- If I was reading the newspaper in the kitchen, then my glasses are on the kitchen table.
- Where are the glasses?

- P = my glasses are on the kitchen table,
- Q = I saw my glasses at breakfast.
- R = I was reading the newspaper in the living room
- S = I was reading the newspaper in the kitchen.
- T = my glasses are on the coffee table.
- U = I was reading my book in bed.
- V = my glasses are on the bed table.

$$1. P \rightarrow Q$$

$$2. R \vee S$$

$\neg T$?

$$3. R \rightarrow T$$

$$4. \neg Q$$

$\neg P$?

$$5. U \rightarrow V$$

$$6. S \rightarrow P$$

$\neg V$?

Proof Structure: Natural Deduction

- A formal proof of p is a tree
 - whose **root** is labeled p and
 - whose **children** are assumptions $p_1, p_2, p_3, \dots$ of the rule r we used to conclude p .
- Natural Deduction:
 - Natural Deduction tries to follow the **natural style of reasoning**.
 - Most of the proof consists of **forward** reasoning, i.e.
 - deriving conclusions,
 - deriving new conclusions from these conclusions, etc.
 - Occasionally hypotheses are introduced or dropped.

Proof Systems

- For a given proof system B ,
 - does $\vdash$ conclude "correct" conclusions from premises ?
 - Can we conclude all true statements with B ?
- Correct with respect to what ?
 - With respect to the semantic definition of the logic.
 - In the case of propositional logic truth tables gives us this.

Boolean Logic as a "Proof" System

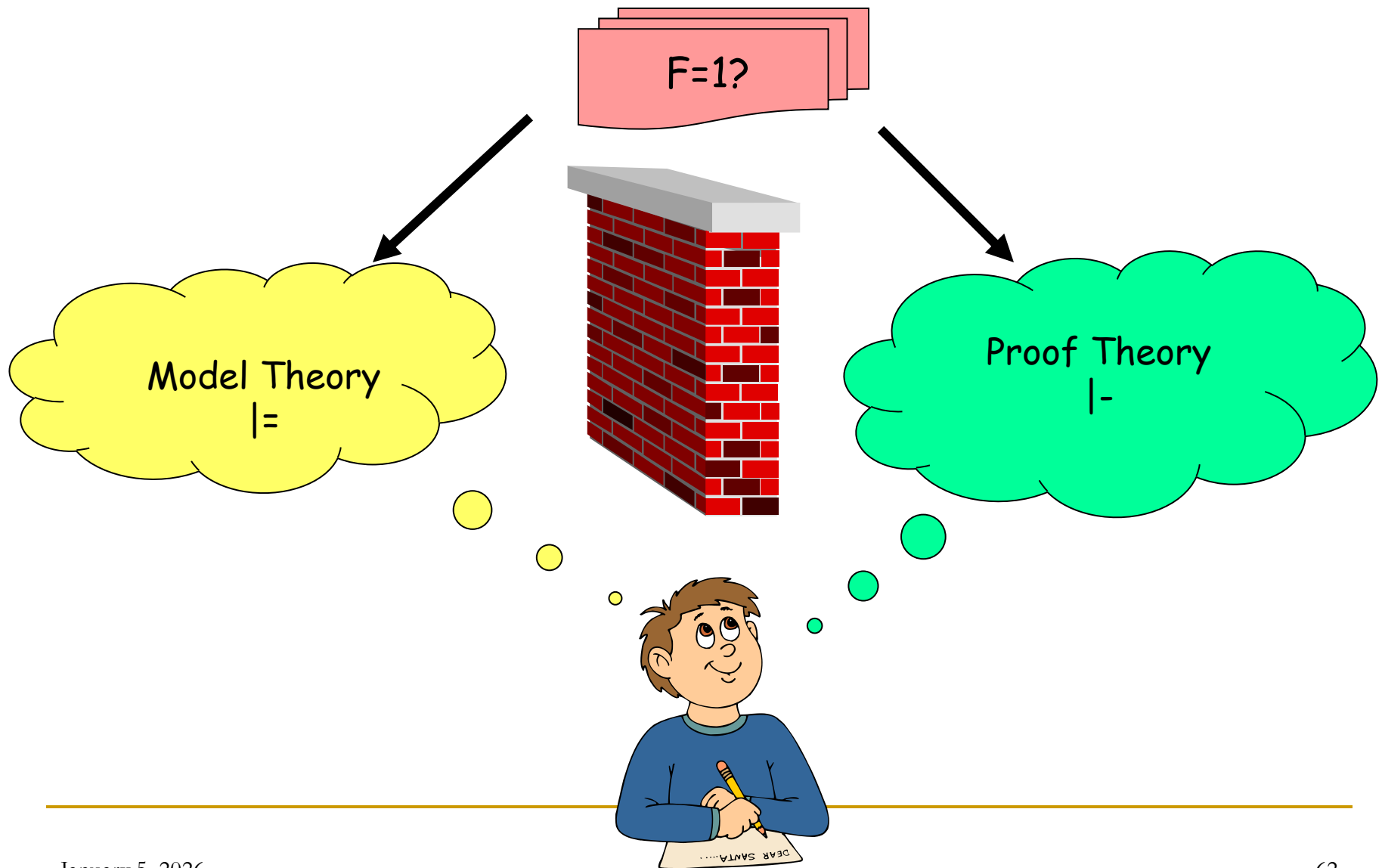
$X + 0 = X, X + 1 = 1$	(Identity)
$X + X = X$	(Idempotent Law)
$X + \bar{X} = 1$	(Complement)
$\overline{\overline{X}} = X$	(Involution Law)
$X + Y = Y + X$	(Commutative)
$X + (Y + Z) = (X + Y) + Z$	(Associative)
$X(Y + Z) = XY + XZ$	(Distributive)
$\overline{X + Y} = \bar{X}\bar{Y}$	(DeMorgan's Law)
$X + XY = X$	(Absorption Law)
$X + \bar{X}Y = X + Y$	(Absorption Law)
$XY + \bar{X}Z + YZ = XY + \bar{X}Z$	(Consensus Theorem)

<u>Expression</u>	<u>Rule(s) Used</u>
$C + \bar{B}\bar{C}$	Original Expression
$C + (\bar{B} + \bar{C})$	DeMorgan's Law.
$(C + \bar{C}) + \bar{B}$	Commutative, Associative Laws.
$T + \bar{B}$	Complement Law.
T	Identity Law.

- Do you believe the result obtained in this proof system as a sound fact?
- Sound in what sense?



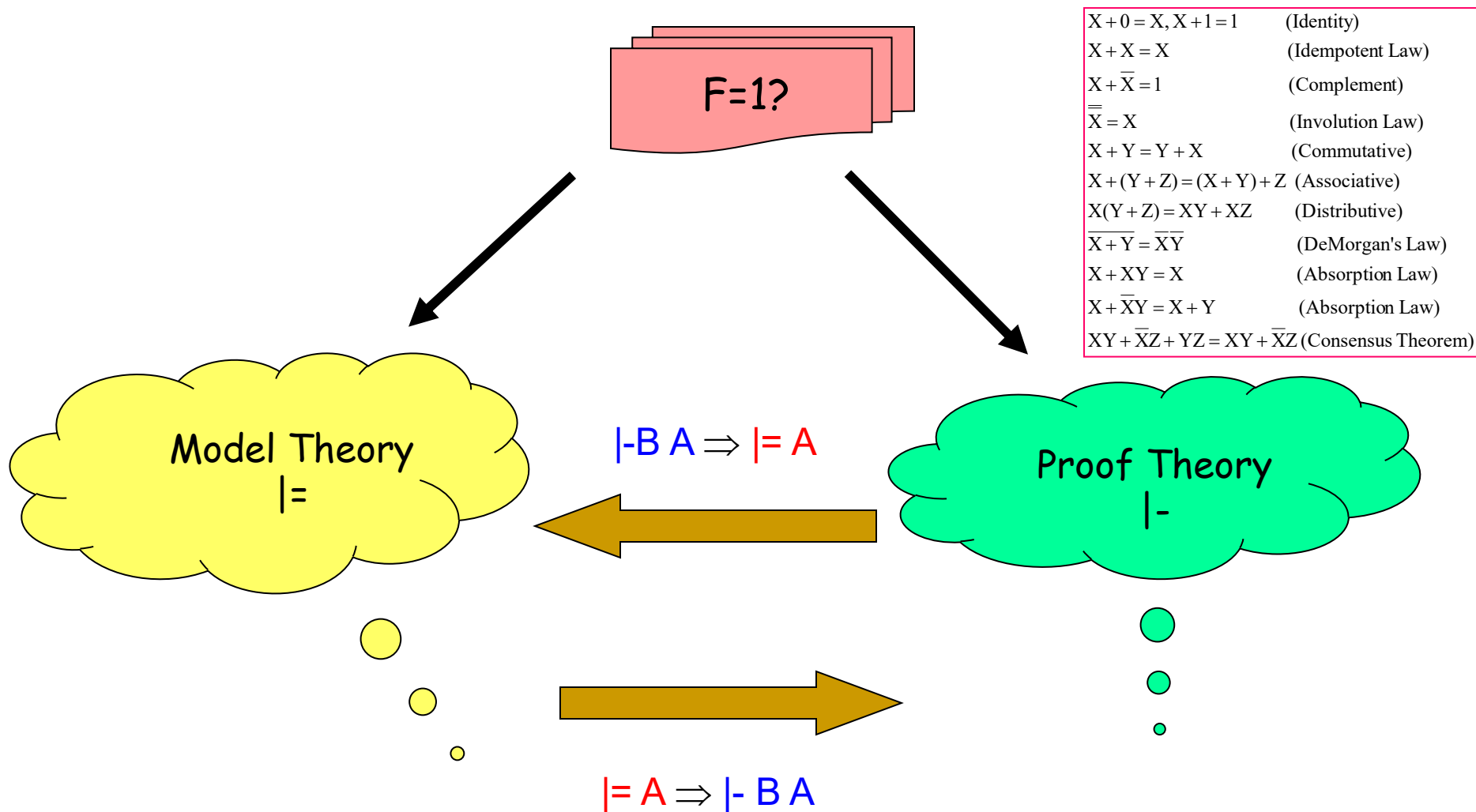
Two Ways to Establish the Truth



Evaluation on Proof Systems

- For a given proof system B ,
- Soundness:
 - all provable formulas (theorems) are logically (semantically) true.
 - nothing that is false may be proven.
 - $\vdash_B A \Rightarrow \models A$
- Completeness:
 - all valid formulas (semantically true) are provable (theorems)
 - everything that is true can be proven.
 - $\models A \Rightarrow \vdash_B A$

Two Ways to Establish the Truth



Boolean Proof Systems

- Theorem.
 - The propositional logic is complete and consistent,
 - There are complete and consistent deduction systems.
- For examples:
 - Hilbert axiom system is sound and complete
 - Resolution is a sound and complete inference system for CNF.
- Given a logic (syntax and semantics), there may be one or more sound proof systems.

To Remember ...

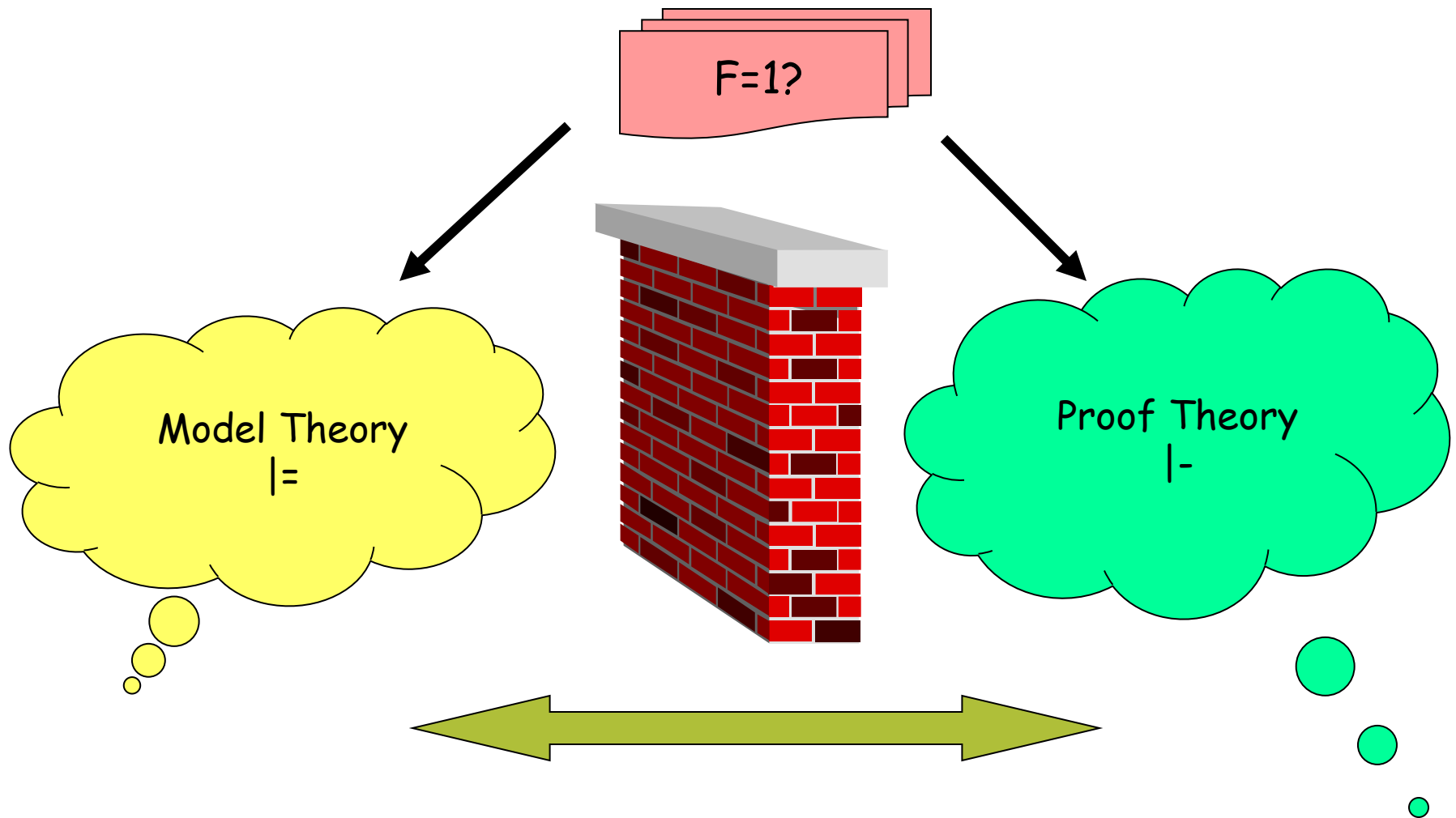
$\vdash P$ means “P is a theorem”

$\models P$ means “P is true”

$P \Rightarrow Q$ means “ $P \rightarrow Q$ is true”

$P \Leftrightarrow Q$ means “ $P \leftrightarrow Q$ is true”

Two Ways to Establish the Truth



Two Ways to Establish the Truth

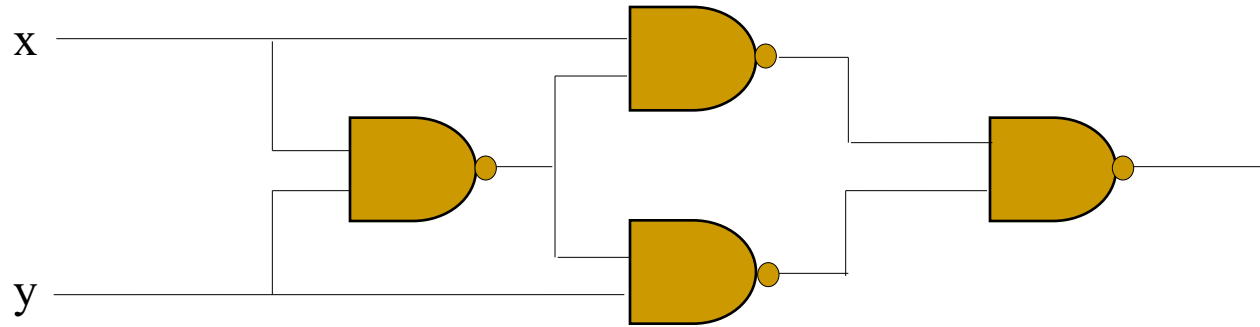
- More generally (beyond propositional logic):
 - Enumeration is possible only in some logics.
 - Deduction cannot necessarily be fully automated.

Summary

- Logic can be used to
 - model and specify the correctness
 - Establish the truth of the correctness:
 - find the truth values.

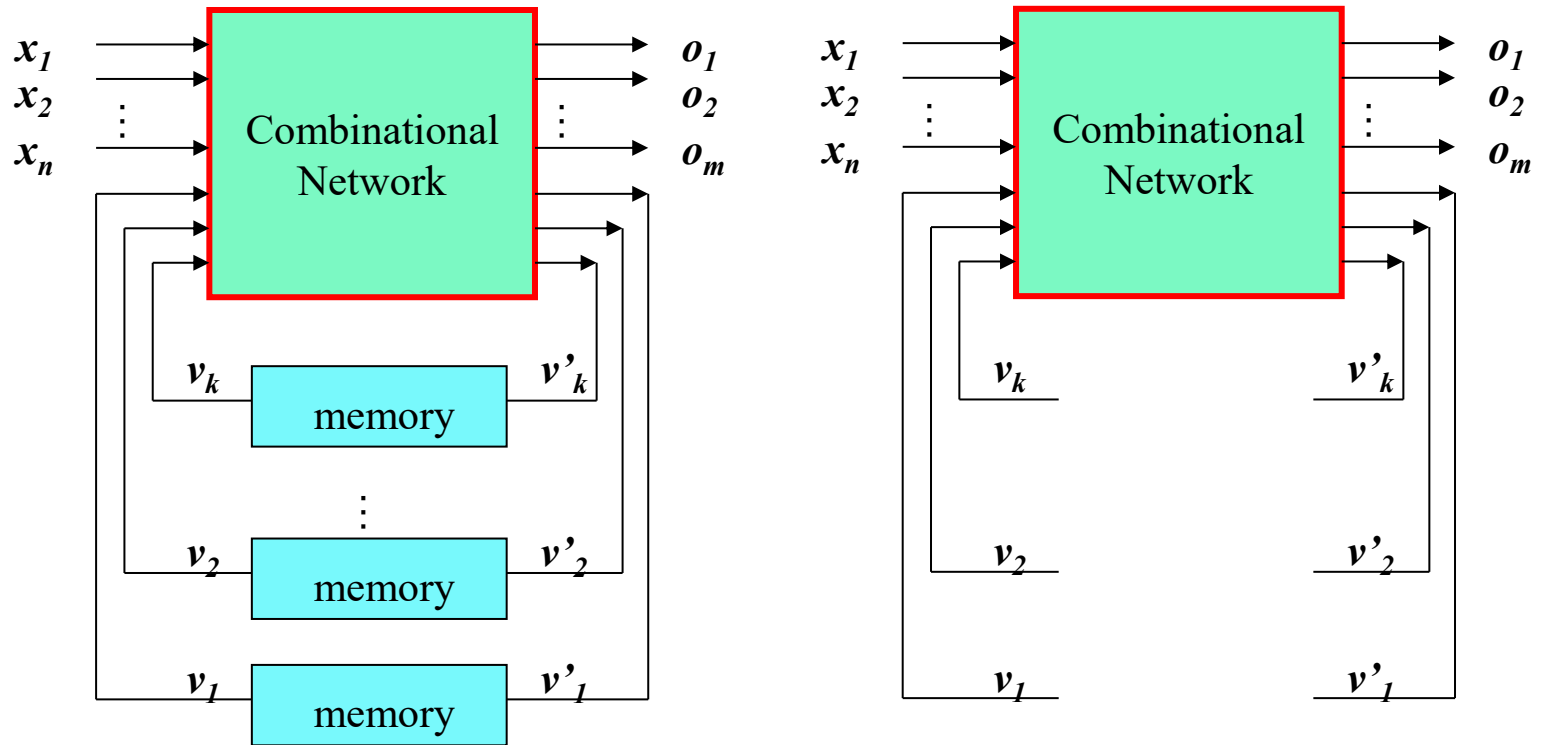
- Question:
 - Is Boolean logic sufficient?
 - Why?

Combinational Circuits



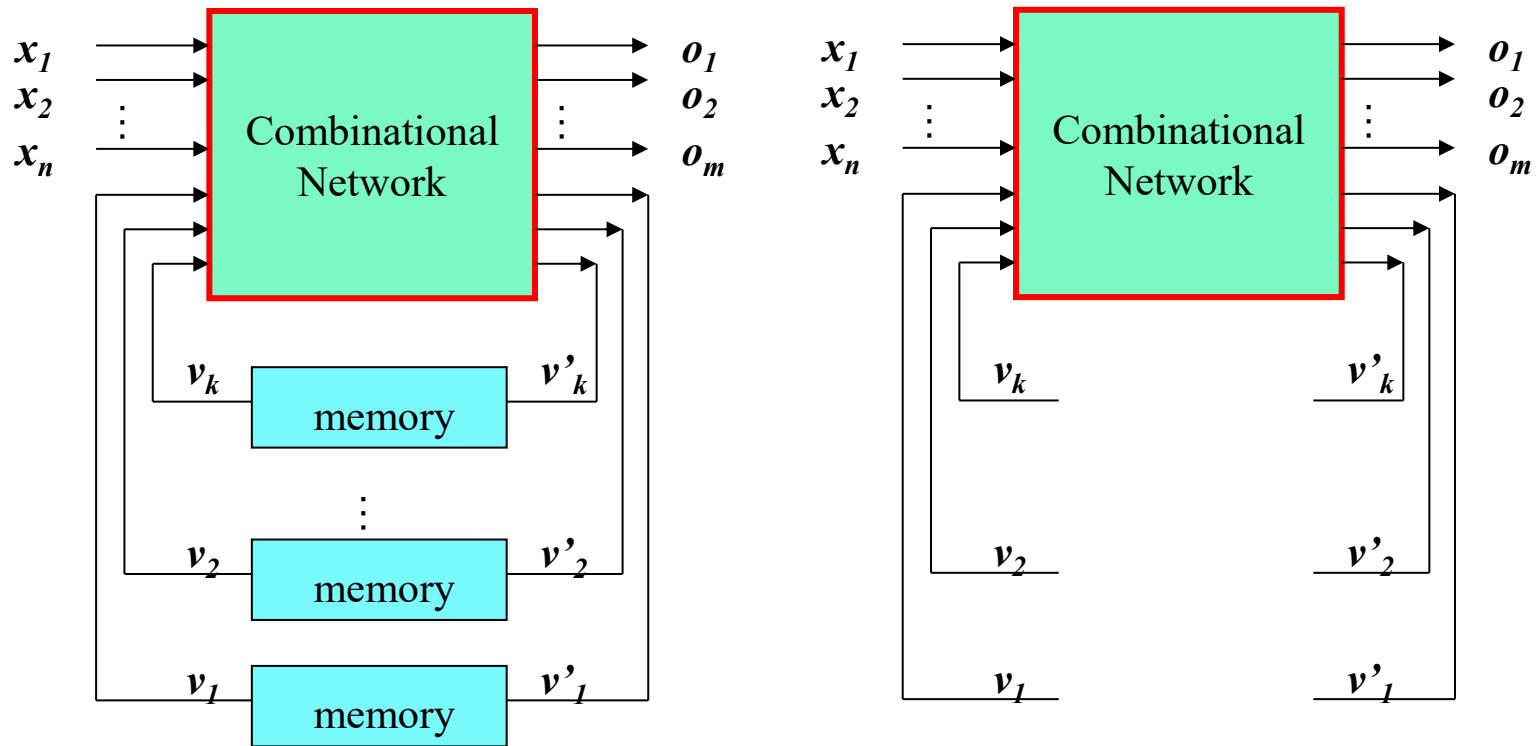
- Boolean logic is sufficient to describe the functional aspect of combinational systems without performance constraints!

Sequential Systems



- How about sequential systems?
 - ❑ Yes for combinational circuits!
 - ❑ How about memory?

Sequential Systems



- In Boolean logic, $v_1 + v'_1$, all the variables are independent in Boolean logic.
- The relation between v_1 and v'_1 is not in the logic, but in our mind.
- Boolean logic is too simple to express it.

How to Extend the Logic?

- How to describe the time or cycles?
- Boolean logic is not enough!
- Possible ways:
 - Extend Boolean logic to have variables to represent time.
 - First-order logic, etc.
 - or
 - Introduce some structure for time.
 - Temporal logics, etc.

Traditional References

- A. Nerode, R.A. Shore. Logic for Applications. Springer-Verlag, 1993.
- U. Schoning. Logic for Computer Science. Birkhauser, 1989.
- D. Duffy. Principles of Automated Theorem Proving. John Wiley & Sons, 1991.
- M. Fitting. First-Order Logic and Automated Theorem Proving. Springer-Verlag, 1990.
- V. Sperschneider, G. Antoniou. Logic: A Foundation for Computer Science. Addison-Wesley, 1991.
- S. Reeves, M. Clarke. Logic for Computer Science. Addison-Wesley, 1991.
- J. H. Gallier. Logic for Computer Science: Foundations of Automated Theorem Proving. Harper & Row, 1986.
- Z. Manna, R. Waldinger. The Deductive Foundations of Computer Programming. Addison-Wesley, 1993.
- C. Chang, R.C. Lee. Symbolic Logic and Mechanical Theorem Proving. Academic Press, 1973.
- D.C. Ince. An introduction to discrete mathematics and formal system specification. Clarendon Press, Oxford.